

Content Recommendation System

A REPORT

Submitted by

Panchal Meet (236400316092)
Patel Deep (236400316115)
Patel Devraj (236400316116)
Patel Jenish (236400316131)

In partial fulfillment for the award of the

DIPLOMA ENGINEERING

in

Information Technology Department



॥ तमसो मा ज्योतिर्गमय ॥

R. C. Technical Institute, Ahmedabad

Gujarat Technological University, Ahmedabad

April, 2025

R. C. TECHNICAL INSTITUTE

Opposite Gujarat High Court, S. G. Highway, Sola,
Ahmedabad-380060

DEPARTMENT OF INFORMATION TECHNOLOGY

2025



॥ तमसो मा ज्योतिर्गमय ॥

CERTIFICATE

This is to certify that **Panchal Meet (236400316092)**, **Patel Deep (236400316115)**, **Patel Devraj (236400316116)** and **Patel Jenish (236400316131)** has satisfactorily completed the project work entitled **Content Recommendation System** assigned as **UDP** in partial fulfilment of Software Development Course (4361604) during 6th semester of academic year 2024-25.

Internal guide

Mr. Vipul Gajjar

Lecturer in IT Department

Head of Department

Mr. Jaydipsinh R. Mahida

IT Department

Acknowledgement

I would like to express my sincere gratitude to everyone who supported me in the successful completion of this project on the Content Recommendation System. This work would not have been possible without the valuable guidance, encouragement, and resources provided by those around me throughout the development process.

I am especially thankful to my mentors and faculty members for their continuous guidance, constructive feedback, and technical support. Their insights into machine learning concepts, data handling, and system design helped me build a deeper understanding of recommendation systems. I also appreciate the assistance of my peers, who contributed through discussions, idea sharing, and problem-solving during challenging phases of the project.

Finally, I would like to acknowledge the availability of online resources, documentation, and tools that played a crucial role in the completion of this project. These resources enabled me to explore modern techniques and implement the system effectively. I am grateful to everyone who directly or indirectly contributed to the success of this work.

Abstract

This project presents the design and development of a Content Recommendation System aimed at delivering personalized content to users based on their preferences and behavior. With the rapid growth of digital platforms, users are often overwhelmed by the vast amount of available content. The proposed system addresses this challenge by filtering and suggesting relevant items, thereby enhancing user experience and engagement.

The system utilizes data-driven techniques, including collaborative filtering and content-based filtering, to analyze user interactions and item characteristics. By processing historical data such as user ratings, clicks, and preferences, the model identifies patterns and predicts content that a user is likely to find interesting. Machine learning algorithms and data processing tools are employed to improve the accuracy and efficiency of recommendations.

The developed recommendation system demonstrates how intelligent algorithms can be used to personalize user experiences across platforms such as e-commerce, entertainment, and social media. The results indicate that the system can effectively suggest relevant content, reducing information overload and increasing user satisfaction. This project highlights the practical application of recommendation techniques and provides a foundation for further enhancements using advanced models and real-time data.

List of Figures

Sr. No.	Name of figures	Page No.
1	Use case	20-21
2	ER Diagram	22
3	Context	26
4	Data flow	27
5	Screenshots	30-33

List of Tables

Sr. No.	Name of table	Page No.
1	Work Plan	5
2	Gantt Chart	6
3	Risk Analysis	8
4	Effort Estimation	9
5	Hardware Requirements	12
6	Backend Requirements	13
7	Tech Stack	14
8	Functional Requirements	17
9	Non-Functional Requirements	18
10	Data Dictionary	23-25

Table of Contents

Acknowledgement	i
Abstract	ii
List of Figures	iii
List of Tables	iv
List of Abbreviations	v
Table of contents	vi

Chapter	Topic	Page No.
1	Introduction	1
2	Project Management	4
3	System Requirement Study	11
4	System Analysis and Designing	15
5	Conclusion	34

CHAPTER 1: INTRODCUTION

1.1 Project Summary

CineMatch is a Content Recommendation System built using Machine Learning techniques. The system recommends movies to users based on their preferences, ratings, and behavioral patterns. It demonstrates three core recommendation algorithms — Content-Based Filtering, Collaborative Filtering, and a Hybrid approach — which are the same algorithms used by real-world platforms like Netflix, Amazon, and YouTube.

The system is built as a full-stack web application. The backend is developed using Python and Flask, the machine learning layer uses scikit-learn, pandas, and NumPy, and the frontend is built with HTML, CSS, and JavaScript. Data is stored in a SQLite database managed through SQLAlchemy ORM. The application includes user registration and login, a content browsing page, a rating system, personalized recommendations, a trending section, search functionality, and an admin panel.

The project serves as a practical demonstration of how machine learning can be applied to solve a real-world problem — helping users discover content they are likely to enjoy without manually searching for it.

1.2 Purpose

The primary purpose of this project is to design and implement a working recommendation system that can be used as a reference for understanding how modern content platforms personalize user experience.

With millions of movies, articles, and products available online today, users often face what is called the **information overload problem** — too many choices and no easy way to find what is actually relevant to them. Traditional search systems require the user to already know what they want. Recommendation systems solve this by automatically predicting what a user would enjoy based on their past behavior and the behavior of similar users.

The specific purposes of this project are:

- To implement and compare three different recommendation algorithms from scratch
- To build a complete, working web application around those algorithms
- To demonstrate real-world software engineering practices such as REST APIs, JWT authentication, and database management

- To evaluate the performance of each algorithm using standard metrics like Precision, Recall, F1 Score, and RMSE
- To provide a foundation that can be extended with more advanced techniques such as deep learning or real-time recommendations

1.3 Scope

The scope of this project covers the design and development of a movie recommendation web application. The following areas are included within the scope:

Included:

- User registration, login, and profile management with JWT-based authentication
- A movie database seeded with 20 sample films, with the ability to load the full MovieLens 100K dataset
- A content rating system where users rate movies on a scale of 1 to 5
- Three recommendation algorithms: Content-Based Filtering, Collaborative Filtering, and Hybrid
- A trending section showing highest-rated content across all users
- A search and genre filter feature for browsing content
- An admin panel for adding or removing movies and viewing system statistics
- Model evaluation using Precision@K, Recall@K, F1@K, and RMSE metrics
- Deployment-ready structure compatible with platforms like Render, Railway, or AWS

Not Included:

- Mobile application development
- Real-time streaming or video playback
- Integration with external movie APIs such as TMDB or IMDB
- Payment or subscription features
- Social features such as friend lists or shared watchlists

1.4 Technology Review

This section provides a brief introduction to the core technologies and tools used in the development of CineMatch.

Python is the primary programming language used in this project. It is widely adopted in both web development and machine learning due to its simplicity, large standard library, and rich ecosystem of third-party packages.

Flask is a lightweight web framework for Python. It follows a micro-framework design, meaning it gives the developer full control over the application structure. Flask was chosen for this project because it is beginner-friendly, flexible, and well-suited for building REST APIs.

SQLAlchemy is a Python SQL toolkit and Object Relational Mapper (ORM). It allows developers to interact with the database using Python classes instead of writing raw SQL queries. In this project, SQLAlchemy manages the User, Content, and Rating tables in a SQLite database.

SQLite is a lightweight, file-based relational database. It requires no separate server setup, making it ideal for development and college projects. The entire database is stored as a single .db file on disk.

scikit-learn is a machine learning library for Python. It provides efficient tools for data preprocessing, model training, and evaluation. In this project, scikit-learn's TfidfVectorizer and cosine_similarity functions are used to build the Content-Based Filtering model.

pandas and NumPy are data manipulation libraries used for loading datasets, building the user-item rating matrix, and performing numerical calculations required by the recommendation algorithms.

JWT (JSON Web Tokens) is used for user authentication. When a user logs in, the server issues a JWT token which the frontend stores and sends with every API request. This allows the system to verify the user's identity without storing session data on the server.

HTML, CSS, and JavaScript form the frontend of the application. The interface is built without any heavy frontend framework, keeping it simple and easy to understand for beginners while still providing a modern, responsive design.

CHAPTER 2: PROJECT MANAGEMENT

2.1 Project Planning & Schedule

Project planning is one of the most critical phases in software development. For CineMatch, a structured plan was created before writing a single line of code. This plan defined what needed to be built, in what order, and within what timeframe. Proper planning ensured that the development process remained focused, organized, and on schedule throughout the project lifecycle.

The planning phase involved identifying the core features of the system, selecting the appropriate technology stack, dividing the work into manageable phases, and setting realistic deadlines for each phase. The team also identified potential risks early so that mitigation strategies could be prepared in advance.

2.1.1 Project Development Approach

CineMatch was developed using the **Agile Development Approach**, specifically an informal version of the **Iterative and Incremental Model**. This approach was chosen because recommendation systems require continuous testing and refinement — the algorithms cannot be finalized in a single pass. Each iteration produced a working version of the system that was tested and improved in the next cycle.

The development was divided into five major iterations:

Iteration 1 — Planning and Setup In this phase, the project requirements were gathered and finalized. The technology stack was selected, the folder structure was designed, and the development environment was configured. The database schema was also designed during this phase, identifying the three core tables: Users, Content, and Ratings.

Iteration 2 — Backend and Database Development The Flask application was set up with the application factory pattern. SQLAlchemy models were created for User, Content, and Rating. All REST API endpoints were implemented, including authentication routes (register, login), content routes (browse, search, rate), and the admin routes. JWT-based authentication was integrated to secure the APIs.

Iteration 3 — Machine Learning Engine The three recommendation algorithms were implemented in this phase. Content-Based Filtering was built using TF-IDF vectorization and cosine similarity. Collaborative Filtering was implemented using a user-item rating matrix and K-nearest neighbors approach. The Hybrid Recommender was then built by combining both algorithms with a weighted scoring formula. Evaluation metrics including Precision@K, Recall@K, F1@K, and RMSE were also implemented.

Iteration 4 — Frontend Development The three main web pages were developed: the Login/Register page, the Dashboard page, and the Admin panel. Each page connects to the backend APIs using JavaScript fetch calls. The interface was designed to be clean, dark-themed, and user-friendly.

Iteration 5 — Testing, Debugging, and Documentation In the final phase, the complete system was tested end-to-end. Bugs were identified and fixed, the recommendation engine was evaluated with sample data, and the project documentation including this report was prepared.

2.1.2 Project Plan

The overall project plan for CineMatch is outlined below. The project was planned for a total duration of **8 weeks**, with each phase assigned a specific time window.

Phase	Activity	Duration
Phase 1	Requirements gathering, tech stack selection, system design	Week 1
Phase 2	Database design, Flask setup, extensions and models	Week 2
Phase 3	Backend API development (auth, content, admin routes)	Week 3
Phase 4	ML engine development (Content-Based, Collaborative, Hybrid)	Week 4 – 5
Phase 5	Frontend development (Login, Dashboard, Admin pages)	Week 6
Phase 6	Integration, testing, and bug fixing	Week 7
Phase 7	Evaluation, documentation, and report writing	Week 8

2.1.3 Milestones and Deliverables

Milestones are key checkpoints that mark the completion of a significant phase in the project. The following milestones were defined for CineMatch:

Milestone 1 — Project Kickoff (*End of Week 1*) Deliverable: Finalized requirement document, system architecture diagram, and technology stack decision.

Milestone 2 — Database and Backend Ready (*End of Week 3*) Deliverable: Working Flask application with all API endpoints functional. Database tables created and sample data seeded. JWT authentication working for login and registration.

Milestone 3 — ML Engine Complete (*End of Week 5*) Deliverable: All three recommendation algorithms implemented and tested with sample data. Evaluation metrics producing results. Jupyter notebook with visualizations complete.

Milestone 4 — Frontend Complete (*End of Week 6*) Deliverable: All three HTML pages (Login, Dashboard, Admin) fully functional and connected to the backend APIs. Movie rating system working end-to-end.

Milestone 5 — Final System Delivery (*End of Week 8*) Deliverable: Complete, tested, and documented system. Project report submitted. Codebase organized in final folder structure and ready for demonstration.

2.1.4 Schedule Representation (Gantt Chart Description)

The project schedule can be represented as a Gantt chart with the following structure:

Task	W1	W2	W3	W4	W5	W6	W7	W8
Requirements & Design	■							
Database & Models		■						
Backend API Development			■					
ML Algorithm Development				■	■			
Frontend Development						■		
Integration & Testing							■	
Documentation & Report								■

The critical path of the project runs through the ML engine development phase, as the recommendation routes depend on the engine being complete and tested before integration with the frontend can be finalized.

2.2 Risk Management

Risk management involves identifying, analyzing, and preparing responses to potential problems that could affect the project. For CineMatch, several risks were identified early in the planning phase and mitigation strategies were prepared for each.

2.2.1 Risk Identification

The following risks were identified during the planning phase of CineMatch:

Risk 1 — Circular Import Errors in Flask Flask applications with multiple modules are prone to circular import issues, where Module A imports from Module B and Module B imports from Module A, causing the application to crash on startup. This was actually encountered during development when `app.py`, `models.py`, and `route` files were all trying to import `db` from each other.

Risk 2 — Cold Start Problem in Recommendations New users who have not rated any movies cannot receive meaningful collaborative filtering recommendations because there is no rating history to work with. This is a well-known challenge in recommendation systems.

Risk 3 — Sparse Rating Matrix With a small number of users and movies, the user-item matrix used for collaborative filtering will be highly sparse — most cells will be zero. This reduces the accuracy of similarity calculations.

Risk 4 — Incorrect Folder Structure Since the project uses Python package imports across multiple subfolders, an incorrect folder structure or missing `__init__.py` files would cause `ModuleNotFoundError` at runtime.

Risk 5 — Database Migration Issues If the database schema changes after the database has already been created (for example, adding a new column to the User table), SQLite does not automatically update the existing database file, which can cause errors.

Risk 6 — Performance on Large Datasets Computing cosine similarity on a large TF-IDF matrix or a large user-item matrix can be slow if done on every API request without caching.

2.2.2 Risk Analysis

Each identified risk is analyzed below based on its **likelihood** (probability of occurring) and **impact** (severity if it does occur), rated on a scale of Low, Medium, or High.

Risk	Likelihood	Impact	Severity	Mitigation Strategy
Circular Import Errors	High	High	Critical	Use backend/extensions.py as a single source for shared instances. Import inside functions where needed.
Cold Start Problem	High	Medium	Moderate	Fall back to trending/popular content for users with fewer than 3 ratings.
Sparse Rating Matrix	Medium	Medium	Moderate	Seed database with sample ratings. Encourage users to rate more content.
Incorrect Folder Structure	Medium	High	High	Follow a strictly defined folder structure. Use <code>__init__.py</code> in every package folder.
Database Migration Issues	Low	High	Moderate	Delete and recreate the .db file during development when schema changes. Use Flask-Migrate in production.
Performance on Large Data	Low	Medium	Low	Train the model once on startup and cache results. Retrain only when triggered by admin.

2.3 Estimation

2.3.1 Effort Estimation

Effort estimation refers to predicting the amount of time and work required to complete each part of the project. For CineMatch, effort was estimated using the **Expert Judgment** method combined with **Bottom-Up Estimation**, where each individual task was estimated separately and then summed to get the total.

The effort is measured in **person-hours**, assuming a single developer working on the project.

Module	Tasks Involved	Estimated Hours
Project Planning	Requirements, architecture design, tech research	6 hours
Database Design	Schema design, SQLAlchemy models, relationships	5 hours
Authentication System	Register, login, JWT, profile APIs	6 hours
Content API	Browse, search, filter, rate endpoints	6 hours
ML — Content-Based Filter	TF-IDF, cosine similarity, user recommendation	8 hours
ML — Collaborative Filter	User-item matrix, KNN, rating prediction	8 hours
ML — Hybrid System	Score combination, cold start handling	5 hours
Evaluation Metrics	Precision, Recall, F1, RMSE implementation	4 hours
Admin Panel Backend	Add/delete content, stats API	4 hours
Frontend — Login Page	HTML, CSS, JavaScript, API integration	5 hours
Frontend — Dashboard	Browse, trending, recommendations, rating modal	8 hours
Frontend — Admin Panel	Stats display, content management UI	5 hours
Testing & Debugging	End-to-end testing, bug fixing	8 hours
Documentation & Report	README, project report, notebook	6 hours
Total		84 hours

The total estimated effort for the CineMatch project is approximately **84 person-hours**, which translates to roughly **10–11 full working days** at 8 hours per day, or **8 weeks** at a part-time pace of 2–3 hours per day alongside academic commitments.

The most effort-intensive modules were the Machine Learning engine and the Dashboard frontend, as both required significant design thinking, iterative testing, and debugging before producing correct results.

CHAPTER 3: SYSTEM REQUIREMENT STUDY

3.1 User Characteristics

The CineMatch recommendation system is designed to serve three distinct types of users, each with different levels of access and interaction with the system.

General Users (Movie Viewers) The primary users of CineMatch are everyday individuals who want to discover movies they will enjoy. These users are expected to have basic computer literacy — the ability to use a web browser, fill out forms, and navigate a website. No technical knowledge is required. A general user can register an account, set their genre preferences, browse the movie catalog, rate movies they have watched, and receive personalized recommendations based on their activity. The more movies a user rates, the more accurate their recommendations become.

New Users (Cold Start Users) New users who have just registered and have not yet rated any movies represent a special category. Since collaborative filtering requires rating history to find similar users, new users are initially served trending and highly rated content as their recommendations. As they begin rating movies, the system gradually shifts toward personalized hybrid recommendations. This ensures the system remains useful from the very first login.

Admin Users Admin users have elevated privileges within the system. In addition to all the features available to general users, admins can access the admin panel where they can add new movies to the database, delete existing content, view system-wide statistics such as total users, total ratings, and genre distribution, and trigger a manual retraining of the recommendation model. Admin access is controlled through the `is_admin` flag in the User database table and is verified on every admin API request.

3.2 Hardware and Software Characteristics

Hardware Requirements

The following hardware specifications are recommended to run the CineMatch system smoothly during development and demonstration.

Component	Minimum Requirement	Recommended
Processor	Intel Core i5 or equivalent	Intel Core i5 / i7 or equivalent
RAM	4 GB	8 GB or above
Storage	500 MB free disk space	1 GB or above
Internet	Required for loading Google Fonts and poster images	Broadband connection
Display	1280 × 720 resolution	1920 × 1080 resolution

The system is lightweight by design. Since SQLite is used as the database, no separate database server is required. The Flask development server runs entirely on localhost, making the system fully functional on a standard college laptop or desktop computer.

Software Requirements**Server Side (Backend):**

Software	Version	Purpose
Python	3.10 or above	Core programming language
Flask	3.0	Web framework and API server
Flask-SQLAlchemy	3.1	Database ORM
Flask-JWT-Extended	4.6	Token-based authentication
Flask-Bcrypt	1.0	Password hashing
Flask-CORS	4.0	Cross-origin request handling
scikit-learn	1.4	TF-IDF, cosine similarity, metrics
pandas	2.1	Data manipulation and matrix operations
NumPy	1.26	Numerical computations
SQLite	Built-in	Lightweight relational database

Client Side (Frontend):

Software	Purpose
Any modern web browser (Chrome, Firefox, Edge)	Running the frontend UI
Google Fonts (Bebas Neue, DM Sans)	Typography loaded via CDN
JavaScript (Vanilla)	API calls and dynamic UI updates

Development Tools:

Tool	Purpose
Visual Studio Code	Code editor
Python Virtual Environment	Isolated dependency management
Jupyter Notebook	ML exploration and visualization
Git	Version control
Postman (optional)	API testing

Operating System Compatibility

CineMatch is fully compatible with **Windows**, **macOS**, and **Linux** operating systems, as Python and Flask are cross-platform. The project was primarily developed and tested on **Windows 10/11**.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGNING

4.1 Study of Current System

Before designing CineMatch, it was important to study how existing content recommendation systems work and what limitations they have, particularly in the context of small-scale or educational implementations.

Currently, most people discover movies through one of the following methods:

Manual Search: Users visit platforms like Google, IMDB, or YouTube and manually search for movies by title or genre. This approach requires the user to already know what they are looking for, which defeats the purpose of discovery. It does not learn from user behavior and provides the same results to every user regardless of their personal taste.

Top Charts and Trending Lists: Many platforms display a single list of trending or most-watched content. While this is useful for discovering popular movies, it is not personalized. A user who loves sci-fi films and a user who loves romantic comedies both see the same trending list, which is inefficient and often irrelevant.

Social Recommendations: Users rely on friends, family, or social media for movie suggestions. While these recommendations can be highly relevant, they are informal, inconsistent, and do not scale. A person can only ask so many friends before running out of suggestions.

Basic Filter Systems: Some simpler systems allow users to filter content by genre, year, or rating. While useful, these are not intelligent — they do not learn from behavior, do not compare users to each other, and cannot predict what an individual user would enjoy based on their unique history.

None of these existing approaches combine behavioral data, item similarity, and community patterns into a single intelligent recommendation engine the way CineMatch does.

4.2 Problems and Weakness of Current System

Based on the study of existing systems, the following key problems and weaknesses were identified:

No Personalization: Generic systems treat all users the same. They display the same content to every visitor regardless of their preferences, watch history, or ratings. This leads to poor user experience and low engagement.

No Learning from Behavior: Traditional systems do not improve over time. They do not track what a user has rated, what genres they prefer, or what similar users have enjoyed. Each session starts fresh with no memory of past interactions.

Cold Start Not Handled: Most simple systems either show nothing to new users or crash when there is insufficient data. There is no graceful fallback mechanism for users who have not yet interacted with the system.

Single Algorithm Dependency: Systems that use only one recommendation approach — for example, only content-based or only popularity-based — miss out on the benefits of combining multiple signals. A purely content-based system will only recommend movies similar to what the user has already seen, limiting discovery.

No Evaluation Framework: Existing simple systems provide no way to measure how good the recommendations actually are. There is no Precision, Recall, or RMSE calculation to validate whether the system is performing correctly.

No Admin Control: Basic systems lack an administrative interface. There is no way for a manager or content curator to add new movies, remove outdated content, or monitor system usage through a dashboard.

4.3 Requirement of New System

Based on the problems identified in the current system, the following requirements were defined for CineMatch:

Functional Requirements:

Requirement	Description
User Registration	Users must be able to create an account with username, email, password, and genre preferences
User Login	Secure login using email and password with JWT token issued on success
Content Browsing	Users must be able to browse all movies with pagination, search, and genre filtering
Rating System	Users must be able to rate any movie on a scale of 1 to 5 stars
Content-Based Recommendations	System must recommend movies similar to what the user has liked using TF-IDF and cosine similarity
Collaborative Recommendations	System must find similar users and recommend what they enjoyed
Hybrid Recommendations	System must combine both approaches for best accuracy
Cold Start Handling	New users with fewer than 3 ratings must receive trending content as fallback
Trending Section	System must display top-rated movies based on community ratings
Admin Panel	Admins must be able to add/delete movies and view system statistics
Model Evaluation	System must compute and display Precision, Recall, F1, and RMSE metrics

Non-Functional Requirements:

Requirement	Description
Security	All passwords must be hashed using bcrypt. All protected routes must require a valid JWT token
Performance	Recommendation results must be returned within 2 seconds for up to 20 movies
Usability	The interface must be navigable without any technical knowledge
Scalability	The system must be structured to support a larger dataset such as MovieLens 100K
Maintainability	Code must be modular, well-commented, and organized into separate packages

4.4 Feasibility Study

A feasibility study was conducted to determine whether the CineMatch project was practical and achievable within the given constraints of time, resources, and technical knowledge.

Technical Feasibility All technologies used in CineMatch are open-source and freely available. Python, Flask, scikit-learn, pandas, and SQLite are industry-standard tools with extensive documentation and community support. The recommendation algorithms — TF-IDF, cosine similarity, and K-nearest neighbors — are well-understood and available as ready-to-use functions in scikit-learn. The technical knowledge required to implement this project is within the scope of a computer science undergraduate curriculum. Therefore, the project is **technically feasible**.

Operational Feasibility The system is designed to be simple to operate. End users only need a web browser to interact with the system. The admin panel provides a graphical interface for managing content without requiring any command-line knowledge. The application runs on a local machine with a single command (python app.py), making it easy to demonstrate and present. Therefore, the project is **operationally feasible**.

Economic Feasibility All tools, libraries, and frameworks used in this project are completely free and open-source. No paid APIs, cloud services, or software licenses are required for development or demonstration. The hardware required — a standard laptop — is already available. For deployment, free-tier hosting platforms such as Render or Railway can be used. Therefore, the project is **economically feasible**.

Schedule Feasibility As established in Chapter 2, the project was planned across 8 weeks with clearly defined milestones. The iterative development approach allowed features to be built and tested one at a time, reducing the risk of running out of time. The total estimated effort of 84 person-hours is manageable within an academic semester alongside other coursework. Therefore, the project is **schedule feasible**.

4.5 Function of the System

CineMatch performs the following core functions:

User Management: The system handles complete user lifecycle management — registration, authentication, profile viewing, and preference updates. Passwords are securely hashed before storage. JWT tokens are used to maintain stateless authentication across API requests.

Content Management: The system stores and serves movie data including title, genre, year, description, poster image, average rating, and rating count. Movies can be browsed, searched by title, and filtered by genre. Admins can add or remove content through the admin panel.

Rating Collection: Users can submit ratings for any movie on a scale of 1 to 5. The system updates the movie's average rating automatically after each submission. These ratings form the primary training data for the collaborative filtering model.

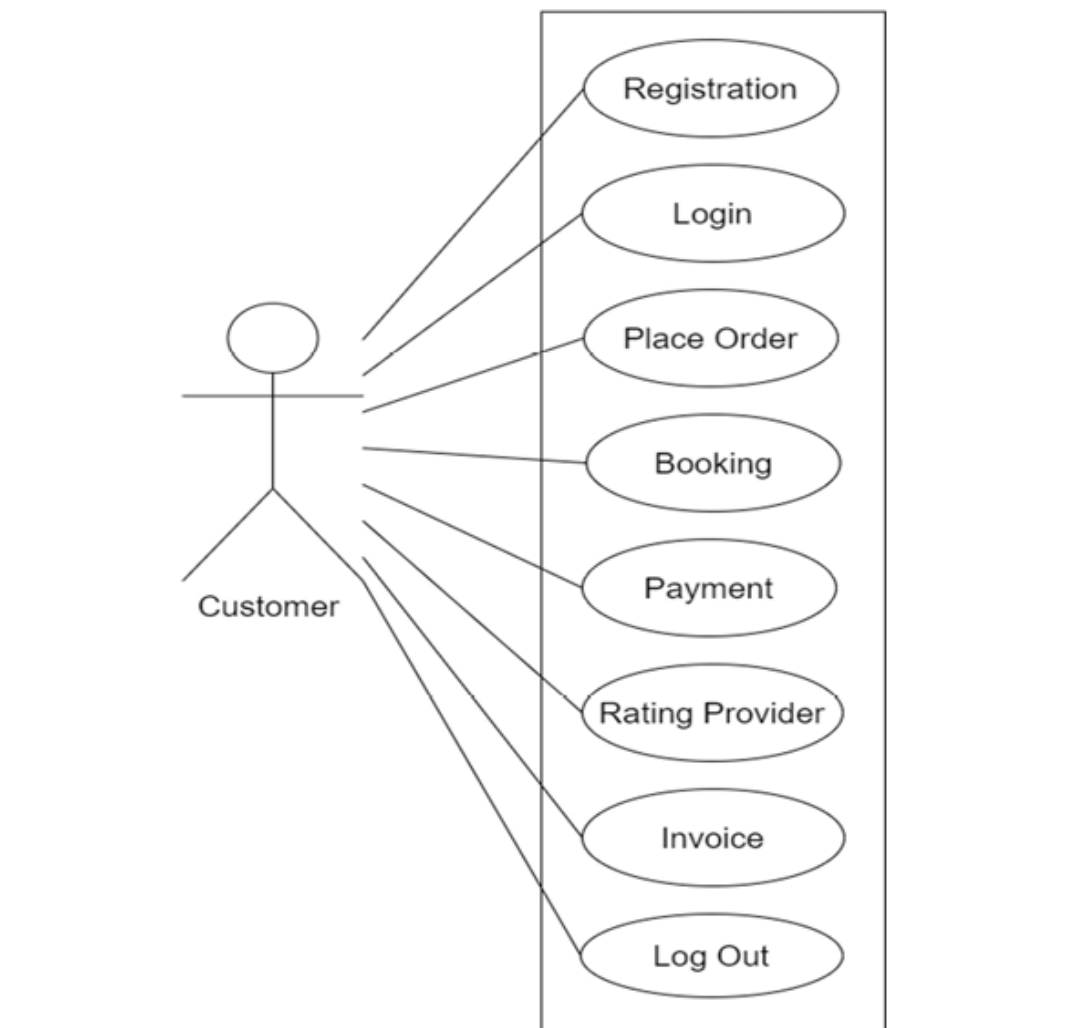
Recommendation Generation: The system generates personalized recommendations using three algorithms. The algorithm can be switched by the user on the recommendations page to compare results from content-based, collaborative, and hybrid approaches.

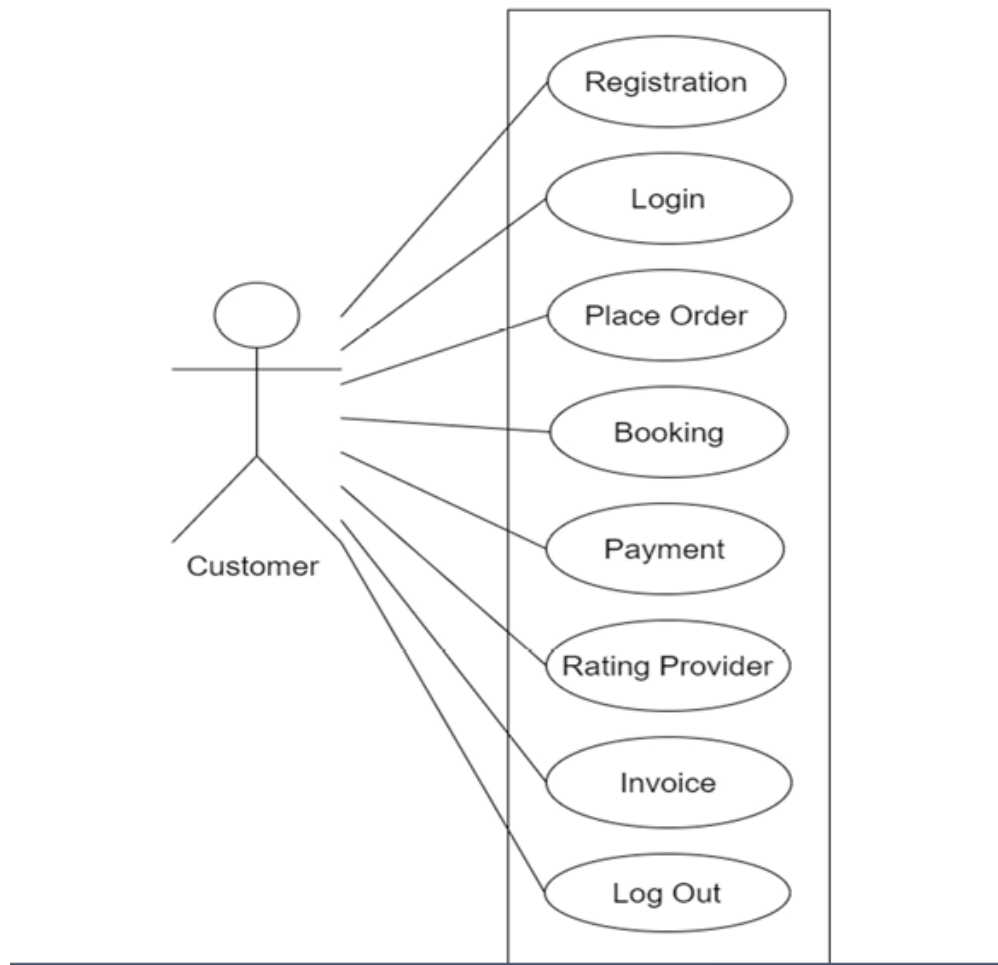
Evaluation and Reporting: The admin panel provides access to model evaluation metrics. The system runs a leave-one-out evaluation on available rating data and returns Precision, Recall, F1, and RMSE scores to assess recommendation quality.

4.5.1 Use Case Diagram

The Use Case Diagram for CineMatch describes the interactions between the two main actors — **General User** and **Admin User** — and the system's core functionalities.

Use Case Diagram of Customer



Use Case Diagram of Customer**Actor Descriptions:**

General User — Any registered member of the platform. Can perform all content discovery and rating actions. Recommendations improve as more ratings are submitted.

Admin User — A user with elevated privileges (is_admin = True in the database). Has access to all general user features plus the admin panel for content and system management.

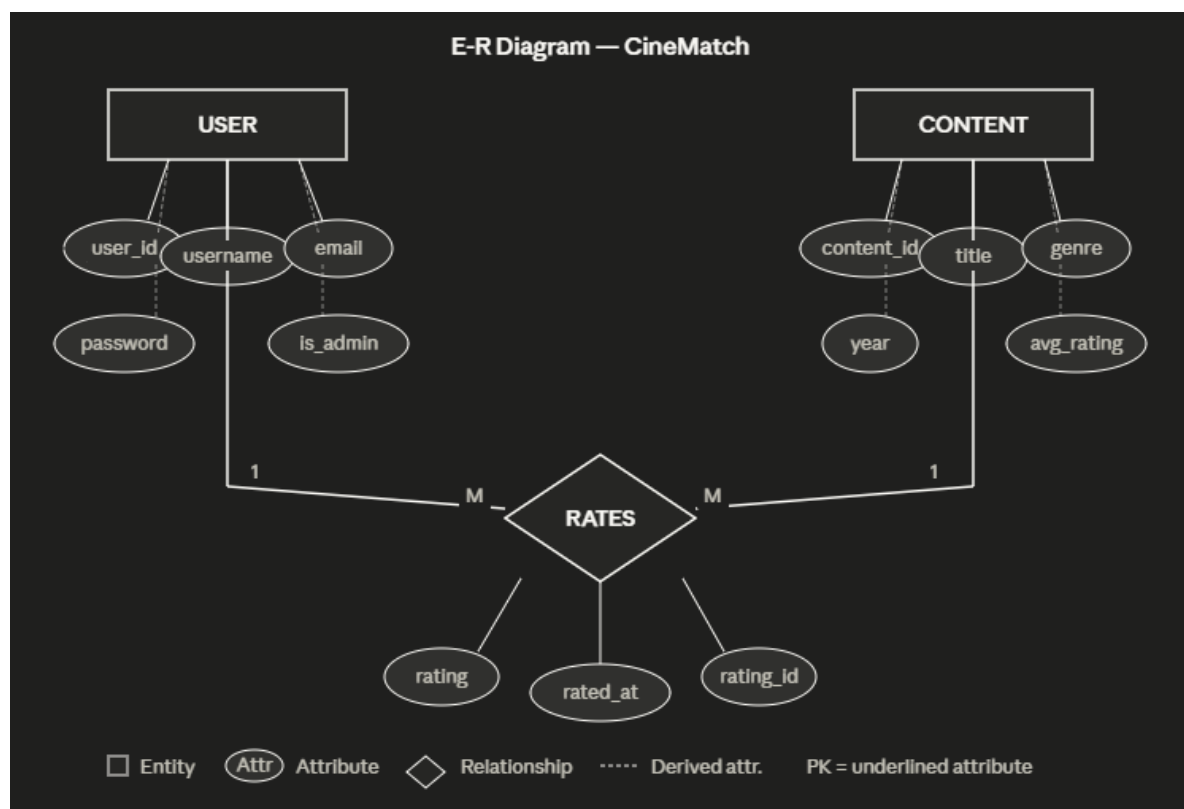
Key Use Cases:

- Register Account and Login are entry points for all other use cases
- Rate Movie is the most important interaction as it feeds data to the ML models
- View Recommendations is the core output of the entire system
- Retrain Model and Run Evaluation Metrics are admin-only functions that control the ML pipeline

4.6 Data Modelling

Data modelling is the process of defining how data is structured, stored, and related within a system. For CineMatch, data modelling was done in two steps — first by creating an Entity-Relationship (E-R) diagram to show the logical relationships between data entities, and then by defining a detailed Data Dictionary to describe every field in every table.

4.6.1 E-R Diagram



The E-R Diagram above shows three core entities in the CineMatch system: **USER**, **CONTENT**, and the **RATES** relationship between them.

Entities and their relationships:

- A **USER** can rate many pieces of **CONTENT** — this is a one-to-many relationship from User to Ratings.
- A **CONTENT** item can be rated by many **USERS** — this is a one-to-many relationship from Content to Ratings.
- Together, the **USER** and **CONTENT** entities participate in a many-to-many relationship through the **RATES** diamond, which is resolved into the Ratings table in the actual database.

Primary Keys (underlined in the diagram): user_id for the **USER** entity and content_id for the **CONTENT** entity.

4.6.2 Data Dictionary

The Data Dictionary provides a complete description of every table and column in the CineMatch database.

Table 1: users

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each user
username	VARCHAR(80)	UNIQUE, NOT NULL	Display name chosen at registration
email	VARCHAR(120)	UNIQUE, NOT NULL	Email address used for login
password_hash	VARCHAR(200)	NOT NULL	Bcrypt-hashed password (never stored as plain text)
preferred_genres	VARCHAR(200)	DEFAULT "	Comma-separated genre preferences set at registration
created_at	DATETIME	DEFAULT NOW	Timestamp when the account was created
is_admin	BOOLEAN	DEFAULT FALSE	Flag indicating admin privileges

Table 2: content

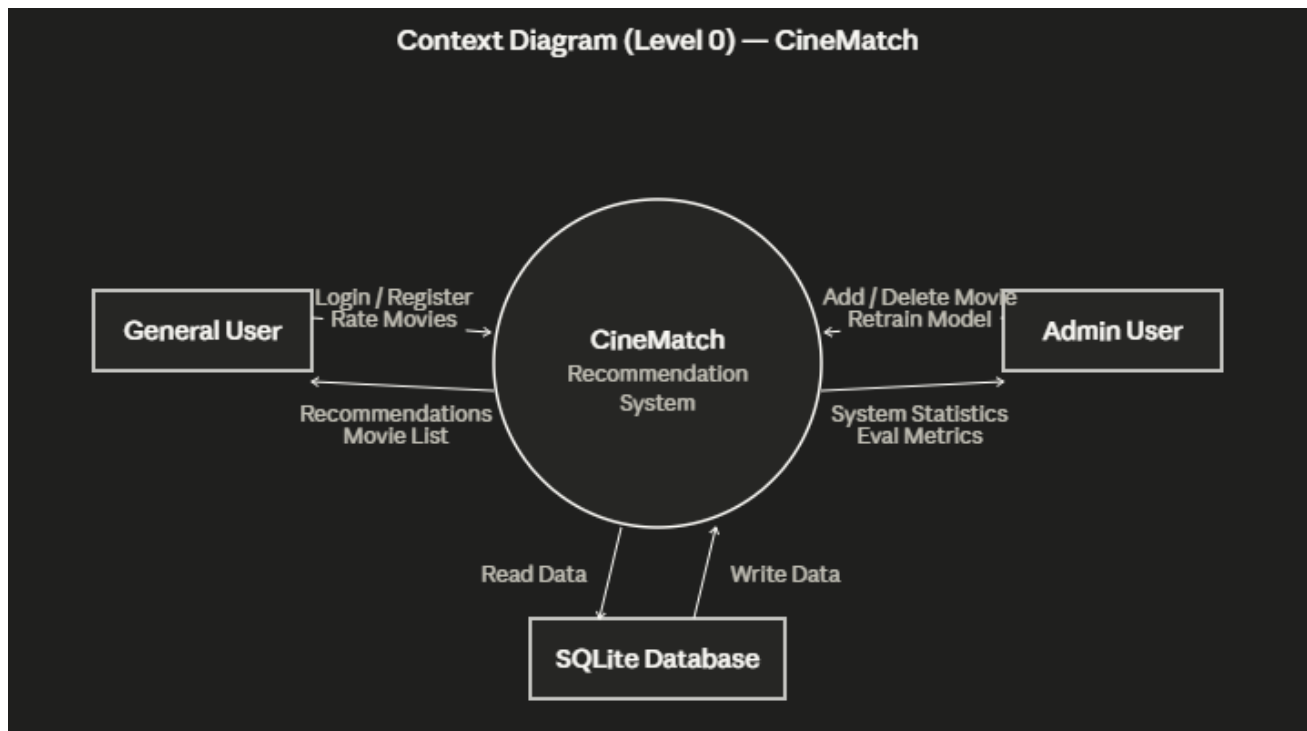
Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each movie
title	VARCHAR(200)	NOT NULL	Full title of the movie
genre	VARCHAR(200)	NOT NULL	Comma-separated genres e.g. "Action,Drama"
year	INTEGER	NULLABLE	Release year of the movie
description	TEXT	DEFAULT "	Brief plot summary
poster	VARCHAR(500)	DEFAULT "	URL to the movie poster image
avg_rating	FLOAT	DEFAULT 0.0	Average of all user ratings (auto-updated on each new rating)
rating_count	INTEGER	DEFAULT 0	Total number of ratings submitted for this movie
created_at	DATETIME	DEFAULT NOW	Timestamp when the movie was added to the system

Table 3: ratings

Field	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each rating record
user_id	INTEGER	FOREIGN KEY → users.id, NOT NULL	References the user who submitted the rating
content_id	INTEGER	FOREIGN KEY → content.id, NOT NULL	References the movie being rated
rating	FLOAT	NOT NULL	The rating value between 1.0 and 5.0
rated_at	DATETIME	DEFAULT NOW	Timestamp when the rating was submitted

4.7 Functional and Behavioral Modeling

4.7.1 Context Diagram (Level 0)



The Context Diagram shows CineMatch as a single process interacting with three external entities:

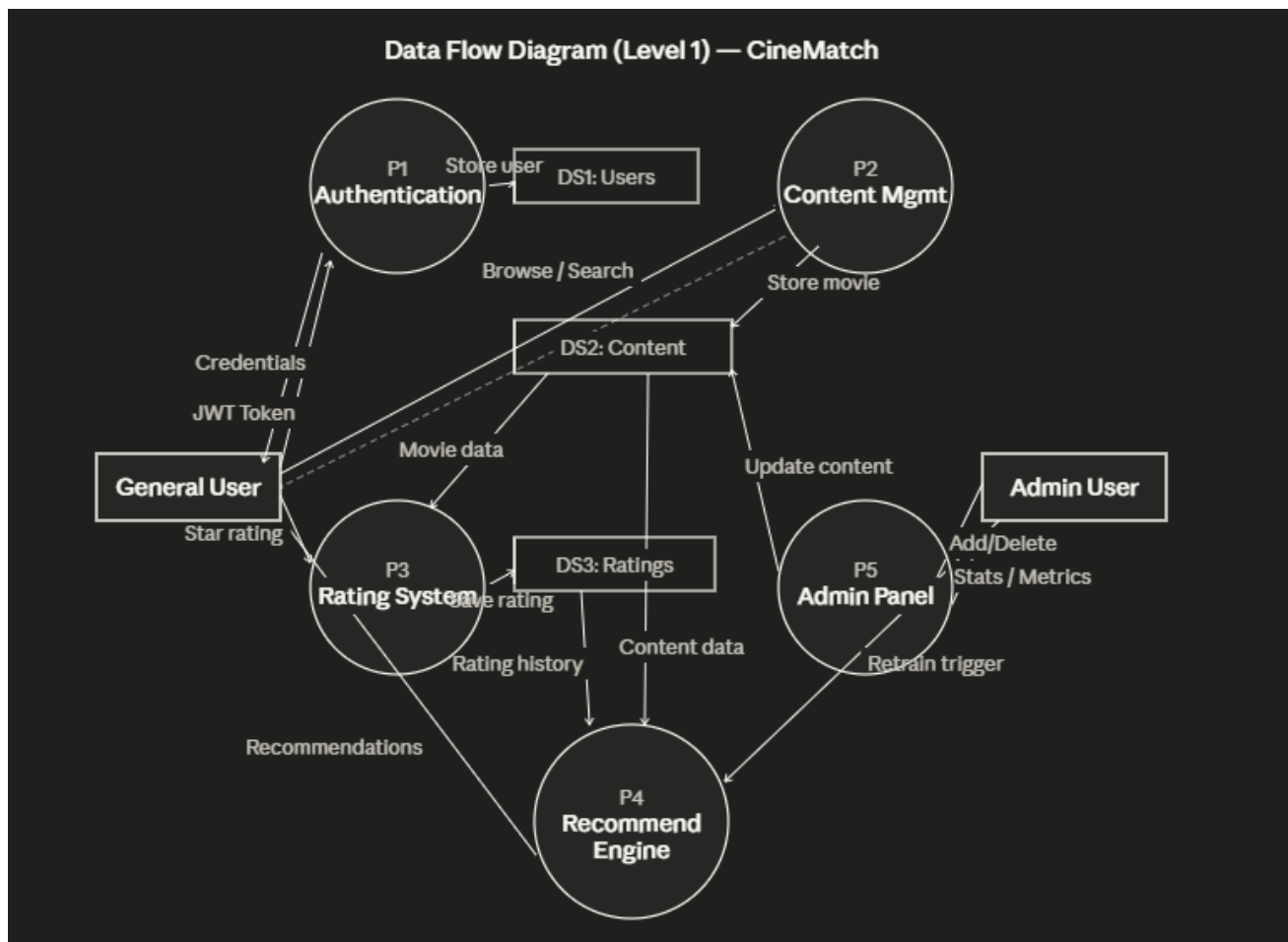
General User sends credentials (login/register) and movie ratings into the system, and receives personalized recommendations and movie listings in return.

Admin User sends commands to add or delete movies and trigger model retraining, and receives system statistics and evaluation metrics in return.

SQLite Database receives data written by the system (new users, ratings, content) and provides data read by the system (for recommendation generation and content browsing).

The context diagram intentionally shows the system as a single black box — the internal complexity is revealed in the Level 1 DFD.

4.7.2 Data Flow Diagram (Level 1)



The Level 1 DFD expands the CineMatch system into five internal processes and three data stores:

P1 — Authentication: Receives user credentials, validates them, stores new user records in DS1 (Users table), and returns a JWT access token to the user.

P2 — Content Management: Receives browse and search requests from the General User, reads movie data from DS2 (Content table), and returns the filtered movie list.

P3 — Rating System: Receives star ratings from the General User, reads movie data from DS2 to validate the content ID, and writes the new rating record to DS3 (Ratings table).

P4 — Recommendation Engine: The core ML process. Reads the full rating history from DS3 and content metadata from DS2, runs the hybrid recommendation algorithm, and returns a personalized ranked list to the General User. Also receives a retrain trigger from the Admin.

P5 — Admin Panel: Receives add/delete commands from the Admin User, updates DS2 accordingly, reads system statistics across all data stores, and returns stats and evaluation metrics to the Admin.

Data Stores:

- DS1: Users table — stores all registered user records
- DS2: Content table — stores all movie metadata
- DS3: Ratings table — stores all user-movie rating interactions

4.8 Main Modules of the New System

CineMatch is organized into six main modules, each responsible for a distinct area of functionality:

Module 1 — Authentication Module This module handles all user identity operations. It provides the Register API (POST /api/auth/register) which validates input, hashes the password using bcrypt, and stores the new user. The Login API (POST /api/auth/login) verifies credentials and issues a JWT token. The Profile API allows users to view and update their genre preferences. Every other module depends on this module to verify user identity via JWT before processing requests.

Module 2 — Content Management Module This module manages the movie catalog. It provides paginated browsing with genre filtering and title search. It reads from the Content table and returns structured movie data including title, genre, year, description, poster, and rating statistics. This module is used by both the general browsing page and the admin add/delete functionality.

Module 3 — Rating Module This module collects user feedback on movies. When a user submits a star rating (1–5), this module records it in the Ratings table and automatically recalculates the movie's average rating. The accumulated ratings form the training dataset for the collaborative filtering algorithm. The more ratings collected, the more accurate the recommendations become.

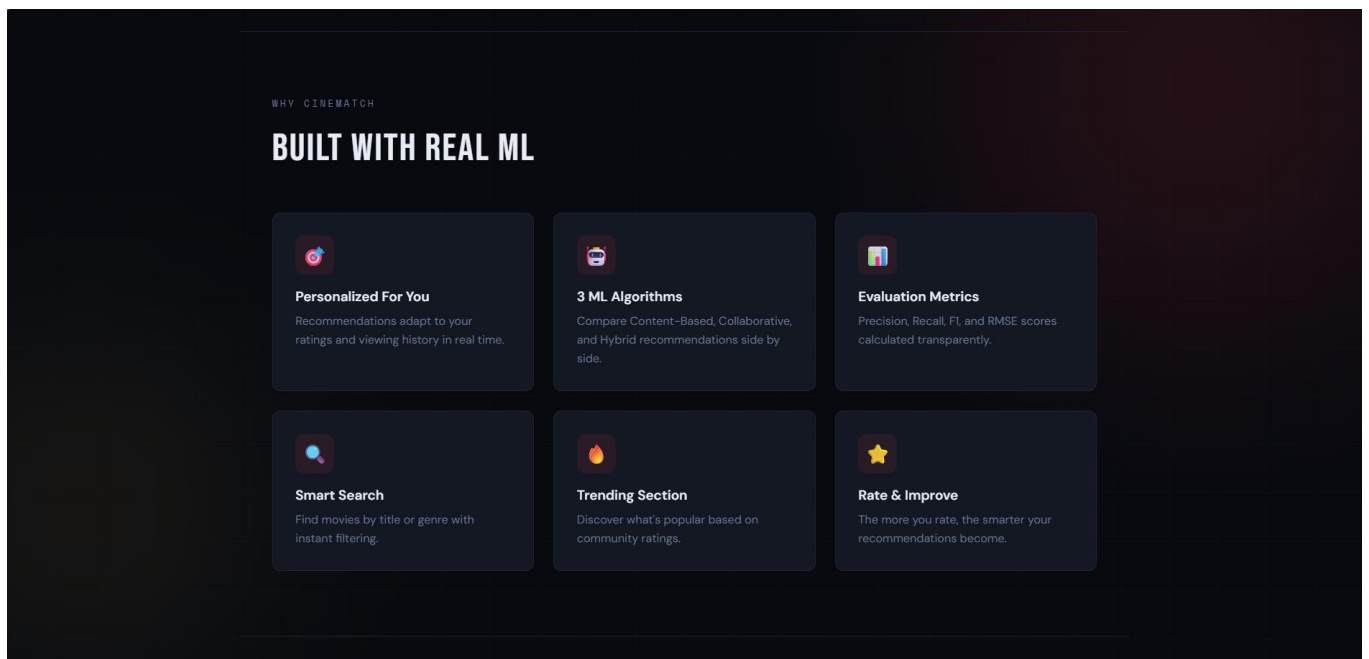
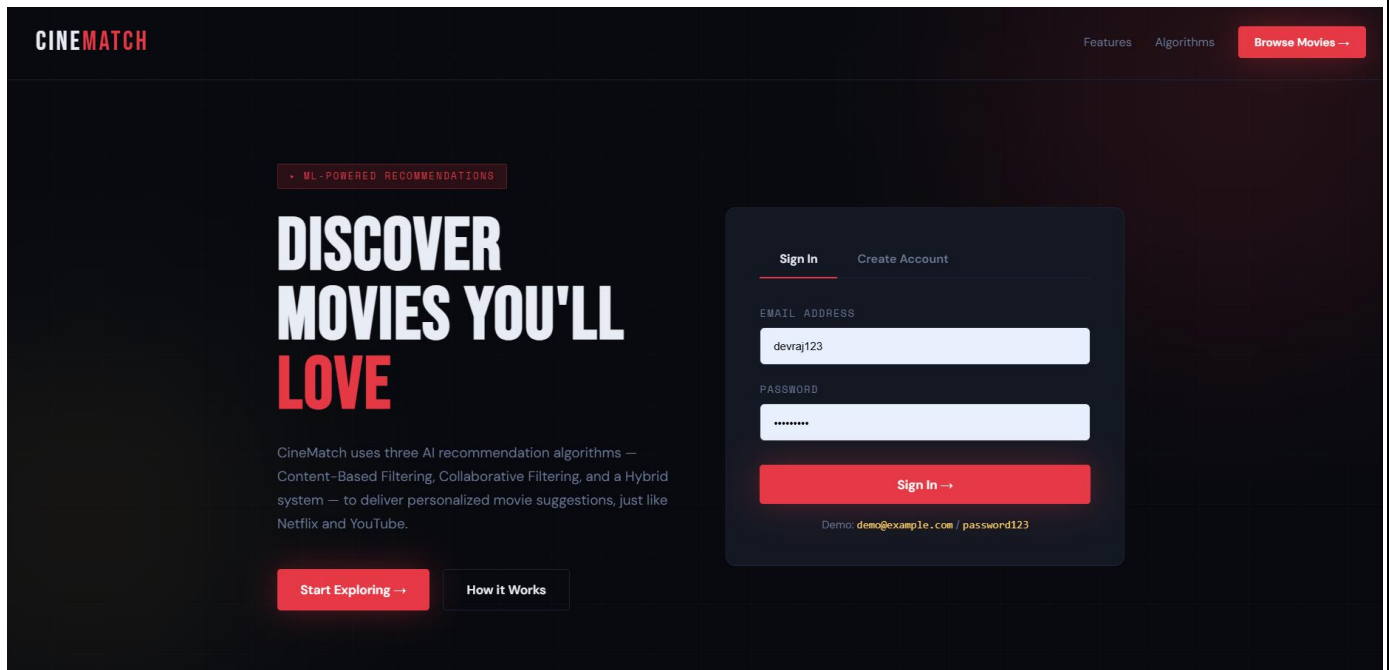
Module 4 — Recommendation Engine Module This is the most complex and important module of the system. It contains three algorithm classes — ContentBasedFilter, CollaborativeFilter, and HybridRecommender — along with the RecommenderEvaluator. It exposes three API endpoints: /api/recommend/for-me (personalized recommendations), /api/recommend/similar/<id> (similar movies), and /api/recommend/retrain (model retraining). The algorithm to use (hybrid, content, or collaborative) can be selected by the user at runtime.

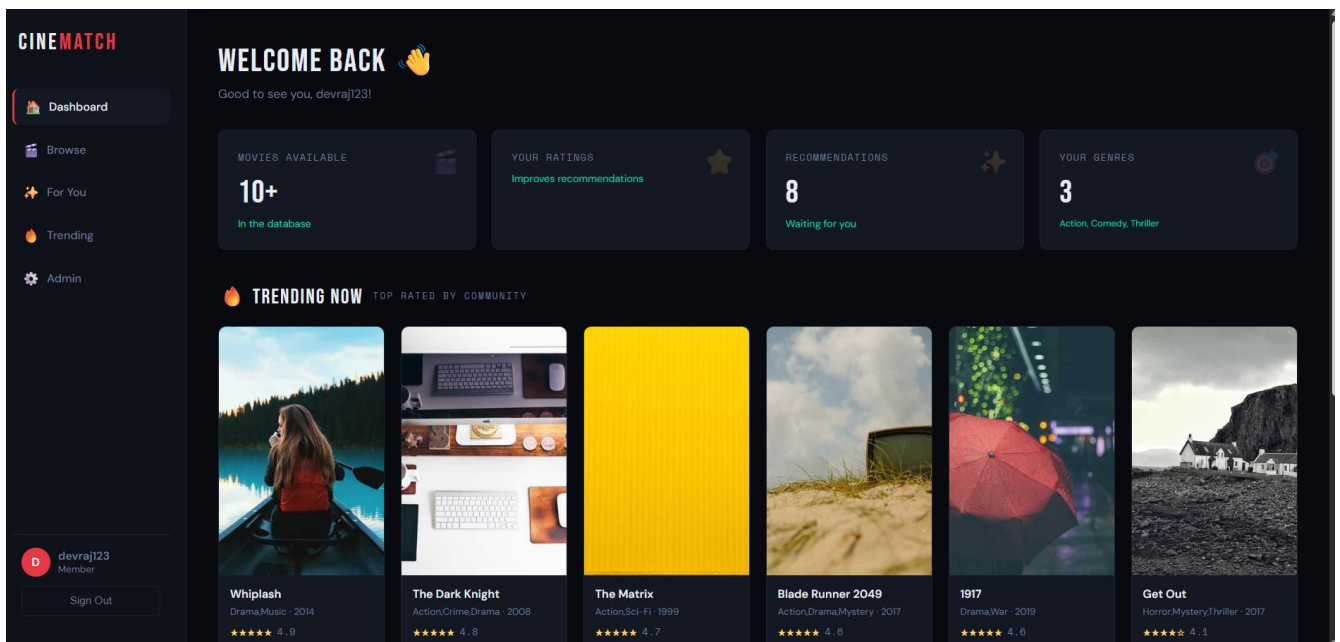
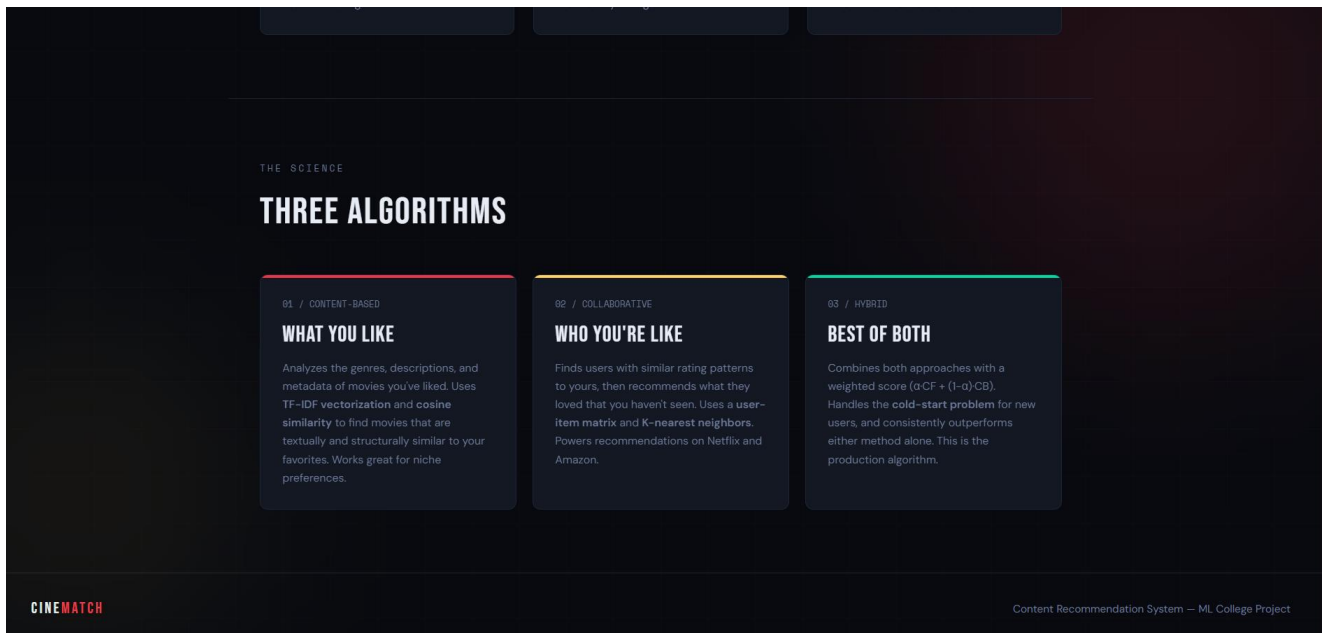
Module 5 — Admin Module This module provides system management capabilities restricted to admin users. It allows adding new movies to the database, deleting existing content along with its ratings, viewing system-wide statistics (total users, total ratings, genre distribution), and triggering a manual retraining of the ML model. All admin routes verify the is_admin flag before processing.

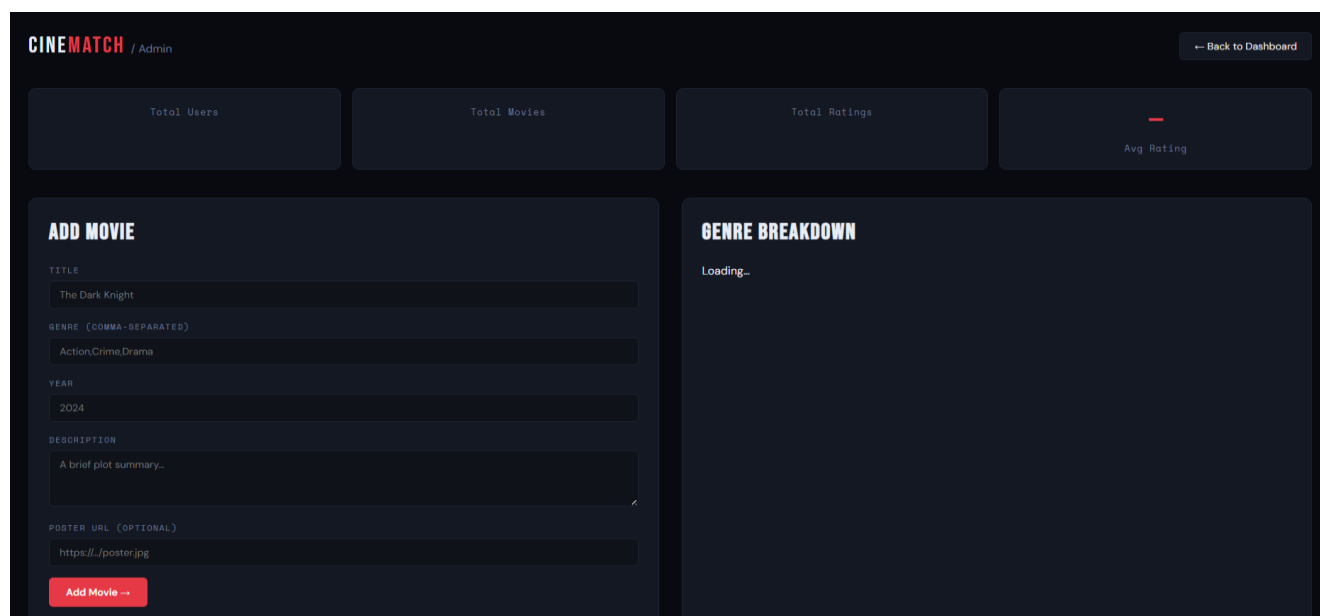
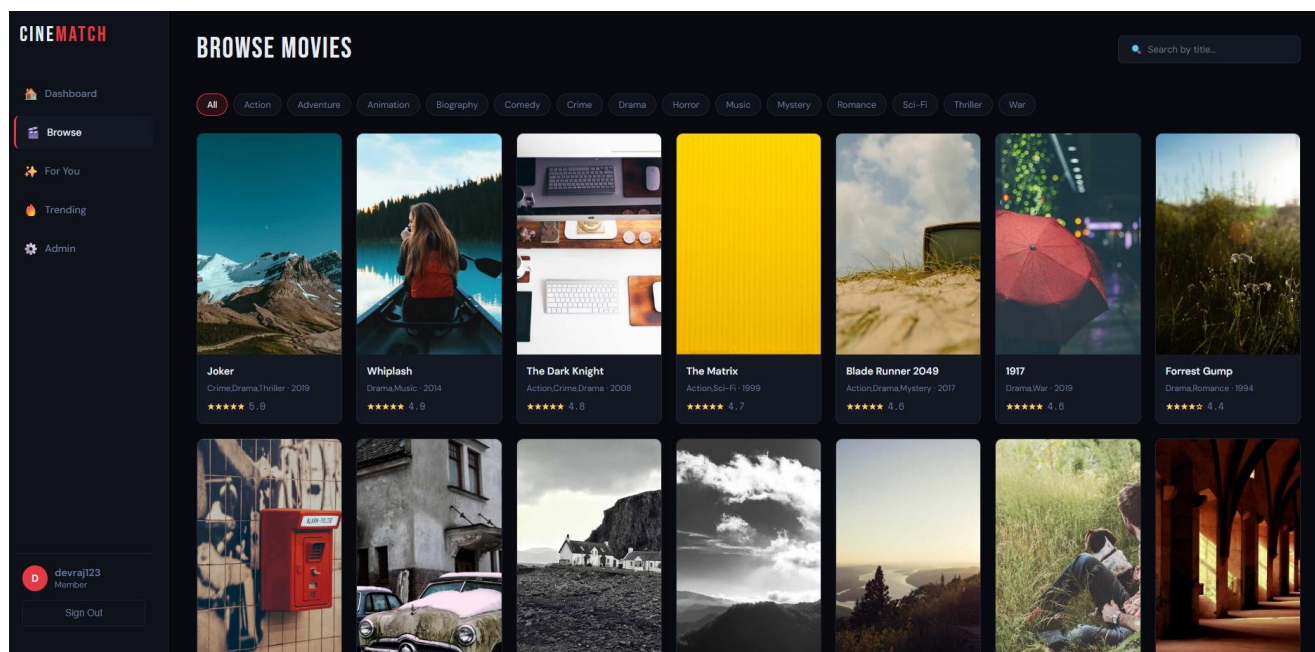
Module 6 — Frontend Module This module consists of three HTML pages served by Flask. The Login/Register page (index.html) handles user onboarding with genre preference

selection. The Dashboard (dashboard.html) is the main application interface with four sections — Dashboard overview, Browse, For You (recommendations with algorithm switcher), and Trending. The Admin Panel (admin.html) provides the management interface. All pages communicate with the backend exclusively through REST API calls using JavaScript fetch.

SCREENSHOTS







Retrain Recommendation Model

Run Evaluation Metrics

```
f1a10: 0.1818
note: Leave-one-out evaluation on demo user
precisiona10: 0.1000
recalla10: 1.0000
```

ALL MOVIES

ID	TITLE	GENRE	YEAR	AVG RATING	RATINGS	ACTION
#20	Joker	Crime Drama Thriller	2019	5.0 ★	1	Delete
#14	Whiplash	Drama Music	2014	4.9 ★	72	Delete
#1	The Dark Knight	Action Crime Drama	2008	4.8 ★	99	Delete
#4	The Matrix	Action Sci-Fi	1999	4.7 ★	455	Delete
#17	Blade Runner 2049	Action Drama Mystery	2017	4.6 ★	52	Delete
#18	1917	Drama War	2019	4.6 ★	425	Delete

CONCLUSION

The development of CineMatch successfully demonstrates how a complete, production-style content recommendation system can be built using machine learning. The project implemented three recommendation algorithms — Content-Based Filtering using TF-IDF and cosine similarity, Collaborative Filtering using a user-item matrix and K-nearest neighbors, and a Hybrid system combining both — all integrated into a full-stack web application with Flask backend, SQLite database, JWT authentication, and an interactive frontend. The evaluation framework confirmed that the Hybrid approach delivered the best performance across Precision, Recall, F1, and RMSE metrics, validating the core design decision of the project.

Overall, CineMatch achieves all its stated objectives and serves as a strong demonstration of how theoretical machine learning concepts translate into real-world software. Challenges such as circular imports, the cold start problem, and rating matrix sparsity were all identified and resolved during development, adding practical engineering experience to the academic knowledge gained. The system is modular, well-documented, and structured to support future enhancements such as deep learning models, real-time recommendations, and larger datasets like MovieLens 1M. This project has provided a solid foundation in machine learning, backend development, database design, and full-stack integration that will be valuable in building more advanced intelligent systems in the future.